WishCraft

Technical Documentation

Personalized greeting cards composited in the browser with HTML5 Canvas, served by a React + Vite stack.

https://drive.google.com/file/d/1kKjNMa-KtpTCADCKFGnYsAxoZaN2R9P/view?usp=sharing

1. Problem-solving Approach

Image overlay logic

The shareable greeting card is generated entirely on the client by compositing layers onto an off-screen HTML5 Canvas. The decision to use canvas (rather than html2canvas-style DOM serialization) keeps the output deterministic across browsers and produces a real PNG blob suitable for download and native sharing.

Rendering pipeline

The CardPreview component drives an async render. On mount and on prop changes (template, user) it resets a ready flag, calls renderCanvas(), and only re-enables the Download / Share buttons once the promise resolves. This guarantees that the user never receives a half-rendered image.

- Background — the CSS gradient string from the template is parsed for hex stops via regex and rebuilt as a canvas linearGradient (0,0 !' W,H).
- Top bar — a semi-transparent black band (rgba(0,0,0,0.55)) is drawn across the top 72px to seat the avatar and the user's name with consistent contrast on any gradient.
- Name — drawn in centered Georgia bold, offset right to leave a 72px square in the corner for the avatar.
- Tag — the template's accent color, sans-serif, left-aligned just under the bar.
- Quote — split on newlines, italic Georgia, drop-shadowed for readability, vertically centered by computing startY from the line count so a 2-line and a 4-line quote both look balanced.
- Watermark — 18% white "WishCraft '(" anchored bottom-right.
- Avatar — drawn last so it always sits on top: ctx.save() + circular clip path, drawImage of the user photo (or a placeholder rectangle + Ø=Üd emoji if no photo), then a 3px green ring stroke.

Async image loading

Profile photos are read from disk via FileReader and stored as data URLs in component state. Because ctx.drawImage cannot accept a data URL directly without an Image element, the render function wraps drawAvatar in img.onload and img.onerror callbacks, both of which call the outer Promise's resolve. This is the lever that gates the ready state.

```
const renderCanvas = () => new Promise((resolve) => {
  drawBase();
  if (user.photo) {
    const img = new Image();
    img.onload  = () => { drawAvatar(img);  resolve(); };
    img.onerror = () => { drawAvatar(null); resolve(); };
    img.src = user.photo;
  } else {
    drawAvatar(null); resolve();
  }
});
```

From canvas to a shareable file

Once rendered, canvas.toBlob("image/png") emits a Blob. URL.createObjectURL turns that Blob into a transient URL that powers two flows: the Download button (anchor tag + programmatic click) and the Share sheet (passes the URL to "Open Image" while social targets use the text via deep links). URLs are explicitly revoked after the download fires and when the share sheet closes, to avoid blob leaks.

2. Tech Stack

Runtime & framework

- React 18.3 — declarative UI, hooks (useState, useRef, useEffect).
- Vite 5.4 — dev server, ES-module bundling, HMR, production build.
- @vitejs/plugin-react — JSX transform and React Fast Refresh.

Browser APIs used directly

- HTML5 Canvas 2D — gradient fills, text, clipping paths, drop shadows, drawImage compositing.
- FileReader — reading user-selected photos as base64 data URLs.
- Image — async decoding of the data URL into a drawable bitmap.
- Blob + URL.createObjectURL / revokeObjectURL — for the download anchor and the open-image share target.
- Clipboard API (navigator.clipboard.writeText) — with a document.execCommand("copy") fallback for non-secure contexts.
- window.open with deep links — WhatsApp (wa.me), Twitter Web Intent, mailto.

Styling

- Inline style objects (CSS-in-JS, no runtime library). Avoids the cost of a styling framework for an app this size and keeps every component self-contained.
- Mixed-script typography — Devanagari and Latin glyphs rendered with Georgia / system-ui via the browser's font fallback chain.

Tooling

- Node.js 24 + npm 11 for the local toolchain.
- pdfkit (dev-only) — used to generate this PDF; not shipped to the browser.

3. Challenges

Race between canvas render and image decode

Initial implementations drew the avatar synchronously, which silently produced cards without a photo when the user's image had not finished decoding. The fix was to make renderCanvas Promise-based and to gate the Download / Share buttons on a ready flag (with a 'Rendering…' overlay over the canvas while opacity ramps up).

Clipboard support in mixed contexts

navigator.clipboard.writeText silently rejects on http:// origins and inside some iframes. A textarea + document.execCommand("copy") fallback was added so the Copy Text action works regardless of context. The button label briefly flips to "Copied! ' " via a 2-second setTimeout to give visual feedback.

Blob URL lifecycle

Object URLs allocated for downloads were not initially revoked, which leaks memory if the user generates many cards in one session. Download URLs are revoked 2s after the click (giving Chromium time to start the download), and share-sheet URLs are revoked when the sheet closes.

Multi-line and bilingual quote layout

Some templates use 2-line English quotes; others use 3- and 4-line Hindi Shayari. A single centered-text fillText call would push longer quotes off the card. Splitting on \n and computing startY = H/2 + 10 " ((lines " 1) × 28) / 2 keeps the block visually centered regardless of how many lines it contains.

Parsing CSS gradient strings

Templates declare backgrounds as CSS linear-gradient strings (convenient for DOM but unusable directly on canvas). A small regex pulls hex stops out of the gradient string and reconstructs them as canvas color stops with a safe fallback if parsing fails.

Premium gating flow

Three states needed to flow correctly: locked template !’ upsell popup !’ preview. The HomeScreen keeps two booleans (showPremium, showPreview) and a selectedTemplate. handleSubscribe simply flips both booleans so the preview opens on the same template the user originally tapped.

4. Future Improvements

Backend & data

- Move the template catalog out of TEMPLATES[] into a CMS / Postgres table with images on S3, so non-engineers can add seasonal templates without a deploy.
- Server-side card rendering (sharp or canvas-on-Node) for higher-quality output, social-preview meta tags, and CDN-cached prerenders.
- Persist generated cards and favorites per user — IndexedDB locally with sync to the backend.

Auth & payments

- Replace the mock login with real OAuth (Google / Facebook) via Firebase Auth or Auth0.
- Wire the Premium plans to Razorpay or Stripe with webhook-driven subscription state; today the subscribed flag is in-memory only.

Performance & scale

- Lazy-load category grids and virtualize long lists once the catalog grows past ~50 templates.
- Use OffscreenCanvas in a Web Worker to keep the main thread responsive while rendering high-resolution cards.
- Code-split the share sheet and the preview overlay so the initial bundle stays under 100 KB gzipped.

Internationalization

- Extract user-facing strings (login, header, button labels) for i18n; many users are Hindi-first.
- Load a Devanagari-optimized font (e.g. Noto Sans Devanagari) for crisper Shayari rendering on the canvas, instead of relying on Georgia's fallback.
- Add support for additional Indic scripts (Tamil, Telugu, Bengali) and right-to-left languages.

Sharing & engagement

- Prefer the Web Share API (navigator.share) when available — it surfaces the native share sheet with the PNG as a real file, instead of a deep link with text.
- Add scheduled greetings (queue a card for a friend's birthday at 9am their local time).
- Animated / video greetings using WebCodecs or a server-side ffmpeg pipeline.

Quality & observability

- Vitest + React Testing Library for component tests, Playwright for end-to-end flows (login !’ premium !’ download).
- PostHog or Mixpanel for funnel analytics — login conversion, template-to-preview rate, subscribe rate.
- Sentry for runtime error tracking, especially on the canvas render path where browser differences bite.

Accessibility & PWA

- Focus management and ARIA roles on the overlays; keyboard navigation through the template grid.
- Ship as a Progressive Web App — service worker for offline templates and an Add-to-Home-Screen prompt; doubles as a path to push notifications for greeting reminders.